МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Построение и анализ алгоритмов»

Тема: Поиск с возвратом.

Студент гр. 4343 _____ Миридаштаки Ф.

Преподаватель _____ Курасов Н. С.

Санкт-Петербург

2026

Цель Работы.

Изучить принцип работы рекурсивного алгоритма поиска с возвратом и решить с его помощью задачу.

Задание.

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до N – 1 и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу – квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков см. на рисунке 1.

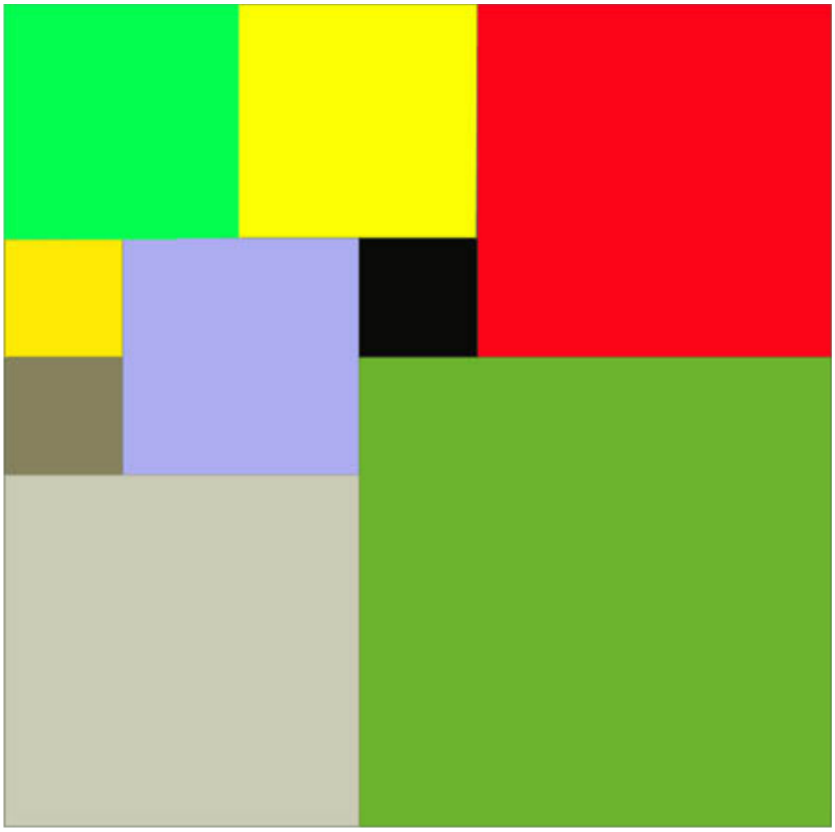

Рисунок 1. Пример построение квадрата N=7

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Теоретический минимум

Поиск с возвратом (backtracking) представляет собой алгоритмический подход, основанный на рекурсивном переборе всех возможных комбинаций. На каждом шаге алгоритм пытается расширить текущее частичное решение, добавляя новый элемент. Если текущий путь не приводит к полному решению или заведомо хуже уже найденного, алгоритм возвращается на шаг назад и пробует следующий вариант.

Этот метод гарантирует нахождение оптимального решения, так как рассматриваются все допустимые конфигурации. Однако из-за экспоненциального роста количества вариантов с увеличением размера задачи, время работы может значительно возрастать. Для ускорения используются различные эвристики и отсечения, которые отбрасывают заведомо бесперспективные ветви поиска на ранних этапах.

Выполнение работы.

Для решения задачи был реализован рекурсивный алгоритм поиска с возвратом на языке C++. В основе алгоритма лежит перебор возможных положений квадратов, начиная с наибольшего размера. Рекурсивная функция go получает текущее количество использованных квадратов и пытается разместить новый квадрат в первую свободную клетку. Если на каком-то этапе количество квадратов становится не меньше уже найденного лучшего решения, ветвь отсекается. При заполнении всего поля решение сравнивается с текущим оптимальным и при необходимости сохраняется.

Перед запуском перебора выполняется проверка четности N. Для четных значений сразу выводится оптимальное решение, состоящее из четырех квадратов размером N/2, что позволяет избежать излишних вычислений.

Оптимизации.

- Выбор размера первого квадрата:

При нечетных значениях N первый квадрат всегда помещается в левый верхний угол. Его размер перебирается от максимально возможного (N-1) до (N+1)/2. Такое ограничение обусловлено тем, что при меньшем размере

первого квадрата оставшаяся область гарантированно не может быть покрыта меньшим количеством квадратов, чем уже известные решения.

- Отсечение заведомо худших ветвей:

Ключевая оптимизация в реализации — проверка на каждом шаге рекурсии: если текущее количество использованных квадратов (used) уже не меньше лучшего найденного решения (best), дальнейший поиск в этой ветви прекращается. Это позволяет отбросить большое количество заведомо неоптимальных вариантов на ранних этапах.

- Перебор размеров от большего к меньшему:

В каждой свободной позиции алгоритм пытается разместить квадраты, начиная с максимально возможного размера и уменьшая его до 1. Такой порядок перебора ускоряет нахождение хороших решений в начале поиска, что усиливает эффективность отсечений.

- Структуры данных:

Для хранения состояния поля используется двумерный вектор vector<vector<bool>> field, где каждый элемент принимает значение true (занято) или false (свободно). Такой подход обеспечивает простой и понятный доступ к любой клетке поля.

Для хранения текущего и лучшего решений используются векторы структур Box, содержащих координаты левого верхнего угла (в 0-индексации) и размер квадрата.

- Оптимизация для четных N:

При четном N алгоритм не запускает рекурсивный перебор, а сразу выдает оптимальное решение из четырех квадратов размером N/2. Это основано на доказанном факте, что для четных N такое разбиение является минимальным.

Сложность алгоритма.

Алгоритм представляет собой полный перебор с возвратом. Временная сложность составляет $O(2^n)$ в худшем случае из-за экспоненциального роста

числа комбинаций. Отсечения сокращают количество рассматриваемых вариантов, но не меняют асимптотику.

По памяти сложность $O(n^2)$ — требуется для хранения двумерной сетки поля размером $n \times n$.

Зависимость кол-ва операции от размера N.

В ходе исследования измерялось количество рекурсивных вызовов функции go для различных значений N.

Для четных N рекурсия не запускается — сразу выдается готовое решение, поэтому количество вызовов равно 1.

Для нечетных N наблюдается экспоненциальный рост числа вызовов с увеличением N. Зависимость близка к $O(2^n)$, где показатель степени зависит от конкретного значения N.

График зависимости количества операций от размера N представлен на рисунке 2.

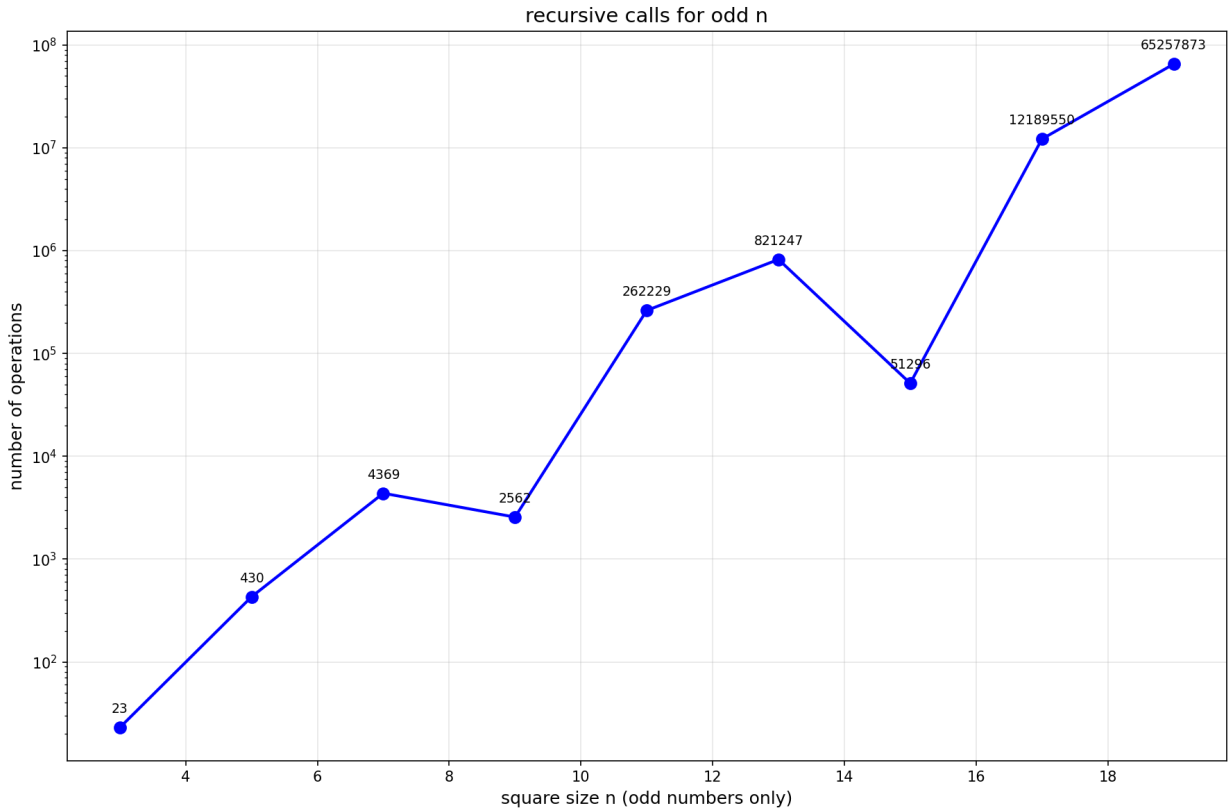

Рисунок 2. Зависимость числа операций от размера квадрата N

Описание функций и структур данных.

Для хранения координат используемых квадратов используется структура Box, хранящая координаты по x и y, а также размер стороны квадрата.

Реализованы следующие структуры и переменные:

int n – размер квадрата

int best – минимальное количество квадратов в лучшем решении

vector<Box> current – данные о текущей вариации расположения квадратов

vector<Box> answer – данные о лучшей конфигурации квадрата

vector<vector<bool>> field – двумерная сетка для отслеживания занятых клеток

long long recursiveCalls – счетчик количества рекурсивных вызовов

long long placementAttempts – счетчик количества попыток размещения квадратов

Основные функции:

bool canPlace(int row, int col, int s) – функция определяет возможность расположения нового квадрата по координатам и размеру стороны в текущей конфигурации

void placeSquare(int row, int col, int s, bool value) – функция устанавливает или удаляет квадрат из текущей конфигурации

bool findFirstEmpty(int& fx, int& fy) – функция ищет первую свободную ячейку для попытки установки квадрата

void go(int used) – основная функция реализующая поиск конфигурации с помощью рекурсии. Функция ничего не возвращает, только сохраняет информацию в глобальных переменных и ими же оперирует. Функция принимает числовое значение used – текущее количество использованных квадратов. Описание работы функции: В самом начале проверяется необходимость продолжения работы алгоритма в данной ветке путем сравнения количества используемых квадратов и уже найденного

минимального решения. Далее ищется первая свободная клетка. Если свободных клеток нет – найдено полное решение, которое сравнивается с лучшим и при необходимости сохраняется. Затем происходит попытка установки нового квадрата в найденную позицию с перебором размеров от максимального до 1. После установки квадрата вызывается та же функция go с увеличенным счетчиком, а после возврата квадрат удаляется.

Код выполнения работы см. в Приложении А.

Приложения

Приложение А

main.cpp:

```
#include <iostream>
#include <vector>
#include <fstream>
#include <chrono>
using namespace std;
using namespace std::chrono;

int n;
int best = 999999;

long long recursiveCalls = 0;
long long placementAttempts = 0;

struct Box {
    int x, y, size;
};

vector<Box> current;
vector<Box> answer;

vector<vector<bool>> field;

bool canPlace(int row, int col, int s) {
    if (row + s > n || col + s > n) {
        return false;
    }
    for (int i = row; i < row + s; i++) {
        for (int j = col; j < col + s; j++) {
            if (field[i][j] == true) {
                return false;
            }
        }
    }
    return true;
}

void placeSquare(int row, int col, int s, bool value) {
    for (int i = row; i < row + s; i++) {
        for (int j = col; j < col + s; j++) {
            field[i][j] = value;
        }
    }
}
```

```
bool findFirstEmpty(int& fx, int& fy) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (field[i][j] == false) {
                fx = i;
                fy = j;
                return true;
            }
        }
    }
    return false;
}

void go(int used) {
    recursiveCalls++;

    if (used >= best) {
        return;
    }

    int row, col;
    if (findFirstEmpty(row, col) == false) {
        if (used < best) {
            best = used;
            answer = current;
        }
        return;
    }

    int maxSize = n - row;
    if (n - col < maxSize) {
        maxSize = n - col;
    }

    if (maxSize > n - 1) {
        maxSize = n - 1;
    }

    for (int s = maxSize; s >= 1; s--) {
        placementAttempts++;

        if (canPlace(row, col, s) == true) {

            placeSquare(row, col, s, true);

            Box b;
            b.x = row;
            b.y = col;
            b.size = s;
            current.push_back(b);
```

```
            go(used + 1);

            current.pop_back();
            placeSquare(row, col, s, false);
        }
    }
}

void runResearch(int maxN) {
    ofstream file("research.csv");
    file << "N,MinSquares,RecursiveCalls,PlacementAttempts,Time_ms\n";

    cout << "\nN\tSquares\tCalls\tAttempts\tTime(ms)\n";
    cout << "----------------------------------------\n";

    for (int testN = 2; testN <= maxN; testN++) {
        n = testN;
        best = 999999;
        recursiveCalls = 0;
        placementAttempts = 0;
        current.clear();
        answer.clear();

        if (n % 2 == 0) {
            auto start = high_resolution_clock::now();

            best = 4;
            recursiveCalls = 1;
            placementAttempts = 0;

            answer.clear();
            int half = n / 2;
            Box b1, b2, b3, b4;
            b1.x = 0; b1.y = 0; b1.size = half;
            b2.x = 0; b2.y = half; b2.size = half;
            b3.x = half; b3.y = 0; b3.size = half;
            b4.x = half; b4.y = half; b4.size = half;
            answer.push_back(b1);
            answer.push_back(b2);
            answer.push_back(b3);
            answer.push_back(b4);

            auto end = high_resolution_clock::now();
            auto ms = duration_cast<milliseconds>(end -
start).count();

            file << n << "," << best << "," << recursiveCalls << ","
                 << placementAttempts << "," << ms << "\n";
```

```
            cout << n << "\t" << best << "\t" << recursiveCalls << 
"\t"
                << placementAttempts << "\t\t" << ms << "\n";
            continue;
        }

        auto start = high_resolution_clock::now();

        field.clear();
        field.resize(n);
        for (int i = 0; i < n; i++) {
            field[i].resize(n, false);
        }

        go(0);

        auto end = high_resolution_clock::now();
        auto ms = duration_cast<milliseconds>(end - start).count();

        file << n << "," << best << "," << recursiveCalls << ","
             << placementAttempts << "," << ms << "\n";

        cout << n << "\t" << best << "\t" << recursiveCalls << "\t"
             << placementAttempts << "\t\t" << ms << "\n";
    }

    file.close();
    cout << "\nData saved to research.csv\n";
}

int main(int argc, char* argv[]) {
    if (argc > 1) {
        string arg = argv[1];
        if (arg == "--research") {
            int maxN = 15;
            if (argc > 2) {
                maxN = atoi(argv[2]);
            }
            runResearch(maxN);
            return 0;
        }
    }

    cin >> n;

    if (n % 2 == 0) {
        auto start = high_resolution_clock::now();

        cout << 4 << endl;
        int half = n / 2;
```

```
        cout << "1 1 " << half << endl;
        cout << "1 " << half + 1 << " " << half << endl;
        cout << half + 1 << " 1 " << half << endl;
        cout << half + 1 << " " << half + 1 << " " << half << endl;

        auto end = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(end - start);

        cout << "\nResearch Data:\n";
        cout << "Recursive calls: 1\n";
        cout << "Placement attempts: 0\n";
        cout << "Time: " << duration.count() / 1000.0 << " ms\n";
        return 0;
    }

    auto start = high_resolution_clock::now();

    field.clear();
    field.resize(n);
    for (int i = 0; i < n; i++) {
        field[i].resize(n, false);
    }

    go(0);

    auto end = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end - start);

    cout << best << endl;
    for (int i = 0; i < answer.size(); i++) {
        cout << answer[i].x + 1 << " "
             << answer[i].y + 1 << " "
             << answer[i].size << endl;
    }

    cout << "Recursive calls: " << recursiveCalls << endl;
    cout << "attempts: " << placementAttempts << endl;
    cout << duration.count() / 1000.0 << " ms\n";

    return 0;
}
```

Приложение В
Тестирование

N = 2
4
1 1 1

1 2 1
2 1 1
2 2 1

N = 3
6
1 1 2
1 3 1
2 3 1
3 1 1
3 2 1
3 3 1

N = 4
4
1 1 2
1 3 2
3 1 2
3 3 2

N = 5
8
1 1 3
1 4 2
3 4 2
4 1 2
4 3 1
5 3 1
5 4 1
5 5 1

N = 6
4
1 1 3
1 4 3
4 1 3
4 4 3

N = 7

9
1 1 4
1 5 3
4 5 2
4 7 1
5 1 3
5 4 1
5 7 1
6 4 2
6 6 2

N = 8
4
1 1 4
1 5 4
5 1 4
5 5 4

N = 9
6
1 1 6
1 7 3
4 7 3
7 1 3
7 4 3
7 7 3

N = 10
4
1 1 5
1 6 5
6 1 5
6 6 5

N = 11
11
1 1 7
1 8 4
5 8 4

8 1 4
8 5 2
8 7 1
9 7 3
9 10 2
10 5 2
11 10 1
11 11 1

N = 12
4
1 1 6
1 7 6
7 1 6
7 7 6

N = 13
11
11
1 1 7
1 8 6
7 8 2
7 10 4
8 1 6
8 7 1
9 7 3
11 10 1
11 11 3
12 7 2
12 9 2

N = 14
4
1 1 7
1 8 7
8 1 7
8 8 7

N = 15

6
1 1 10
1 11 5
6 11 5
11 1 5
11 6 5
11 11 5

N = 16
4
1 1 8
1 9 8
9 1 8
9 9 8

N = 17
12
1 1 9
1 10 8
9 10 2
9 12 4
9 16 2
10 1 8
10 9 1
11 9 3
11 16 2
13 12 1
13 13 5
14 9 4

N = 18
4
1 1 9
1 10 9
10 1 9
10 10 9

N = 19
13

1 1 13
1 14 6
7 14 6
13 14 2
13 16 4
14 1 6
14 7 6
14 13 1
15 13 3
17 16 1
17 17 3
18 13 2
18 15 2

N = 20
4
1 1 10
1 11 10
11 1 10
11 11 10